



Instituto Superior de Engenharia

Politécnico de Coimbra

DEPARTAMENTO DE / DEPARTMENT OF
INFORMÁTICA E SISTEMAS

SimGarden – Simulador de jardim

Trabalho prático no âmbito da
**Unidade Curricular de Programação Orientada a
Objetos**

Autores / Authors

Celso André Ferreira Jordão

2003008910

Coimbra, Janeiro de 2026



INSTITUTO POLITÉCNICO
DE COIMBRA

INSTITUTO SUPERIOR
DE ENGENHARIA
DE COIMBRA

Simulador de Jardim – Relatório Técnico (Meta 2)

Este relatório descreve a implementação do **Simulador de Jardim** (Meta 2), desenvolvido no âmbito da unidade curricular de Programação Orientada a Objetos (2025/2026).

O sistema permite gerir um jardim virtual através de comandos textuais, suportando a criação de uma grelha 2D sem recurso a `std::vector`, a evolução temporal das plantas, a aplicação de ferramentas e um conjunto de validações e limites por turno.

São apresentadas a arquitetura em camadas (MVC adaptado), as principais estruturas de dados e decisões de design (incluindo padrões Factory/Template Method/Strategy e princípios SOLID), bem como um conjunto de testes representativos.

Palavras-chave: C++17, POO, herança, polimorfismo, gestão de memória, simulação, validação de comandos.

This report describes the implementation of the **Garden Simulator** (Milestone 2), developed for the Object-Oriented Programming course (2025/2026).

The system manages a virtual garden through textual commands, including a 2D grid implemented without `std::vector`, time progression for plants, tool application, and comprehensive input validation with per-turn limits.

The document presents the overall architecture (adapted MVC), the main data structures, key design decisions (Factory/Template Method/Strategy, SOLID), and a representative set of validation tests.

Keywords: C++17, OOP, inheritance, polymorphism, memory management, simulation, command validation.

“Simplicidade é a sofisticação máxima.”

— Leonardo da Vinci

ÍNDICE

Resumo	i
Abstract	ii
Epígrafe	iii
Dedicatória	1
Agradecimentos	1
Índice	1
Índice de tabelas	3
Índice de figuras	4
Lista de abreviaturas	5
Lista de siglas e acrónimos	6
Lista de símbolos	7
1 Introdução	8
1.1 Contexto	8
1.2 Objetivos	8
1.3 Organização do Relatório	8
2 Arquitetura e Estruturas de Dados	9
2.1 Arquitetura do Sistema	9
2.1.1 Visão Geral	9
2.1.2 Hierarquia de Classes	9
2.2 Estruturas de Dados Principais	10
2.2.1 Jardim: grelha 2D sem std::vector	10
2.2.2 Posição: célula individual	11
2.2.3 Gestão de memória com smart pointers	11

3	Implementação, Design e Validação	13
3.1	Implementação das Funcionalidades	13
3.1.1	Loop principal: <code>avanca()</code>	13
3.1.2	Comportamento das plantas	13
3.2	Ferramentas	14
3.2.1	Factory Pattern	14
3.2.2	Aplicação de ferramentas (polimorfismo)	14
3.3	Limites por turno	15
3.4	Decisões de design	15
3.4.1	Padrões de design	15
3.4.2	Princípios SOLID (síntese)	16
3.5	Renderização do jardim	16
3.6	Testes e validação	16
3.6.1	Compilação	16
3.6.2	Casos de teste (exemplos)	17
3.6.3	Validações implementadas	17
3.7	Estatísticas e estado da Meta 2	17
3.7.1	Métricas de código (aprox.)	17
3.7.2	Funcionalidades implementadas	18
3.8	Dificuldades e melhorias	18
3.8.1	Dificuldades encontradas (síntese)	18
3.8.2	Melhorias futuras	18
4	Conclusão	19
4.1	Objetivos alcançados	19
4.2	Aprendizagens	19
4.3	Palavras finais	19
	Referências bibliográficas	20
	Anexos	21
	Anexo A - Comandos Disponíveis	22
	Anexo B - Estrutura de Ficheiros	23

ÍNDICE DE TABELAS

3.1	Limites por Turno	15
3.2	Validações e Mensagens de Erro	17
3.3	Estatísticas do Código	17
4.1	Lista completa de comandos	22

ÍNDICE DE FIGURAS

LISTA DE ABREVIATURAS

IEEE *Institute of Electrical and Electronics Engineers*

ISEC Instituto Superior de Engenharia de Coimbra

LISTA DE SIGLAS E ACRÓNIMOS

CD *Compact Disc*

ONU *Organização das Nações Unidas*

LISTA DE SÍMBOLOS

kN Quilonewton

ε_{ax} Extensão axial (%)

1 INTRODUÇÃO

1.1 Contexto

Este relatório documenta a implementação do **Simulador de Jardim**, trabalho prático da unidade curricular de Programação Orientada a Objetos do ano letivo 2025/2026.

O simulador permite ao utilizador gerir um jardim virtual através de comandos textuais, controlando um jardineiro que pode plantar, colher, usar ferramentas e observar o crescimento e morte das plantas ao longo do tempo simulado.

1.2 Objetivos

- Aplicar princípios de Programação Orientada a Objetos (POO);
- Implementar hierarquias de classes com herança e polimorfismo;
- Gerir memória dinâmica de forma segura;
- Utilizar a biblioteca standard de C++ (sem bibliotecas externas);
- Respeitar as restrições do enunciado (sem `std::vector` para a grelha).

1.3 Organização do Relatório

O relatório está organizado em secções que cobrem arquitetura, estruturas de dados, implementação, decisões de design, testes e conclusão.

2 ARQUITETURA E ESTRUTURAS DE DADOS

2.1 Arquitetura do Sistema

2.1.1 Visão Geral

O sistema segue uma arquitetura em camadas baseada num padrão MVC adaptado:

- **Interface** — leitura e validação de comandos do utilizador;
- **Simulator** — controlador central (coordena toda a lógica);
- **Modelo** — jardim, jardineiro, plantas e ferramentas.

2.1.2 Hierarquia de Classes

Classes base abstratas (Plantas)

```

1 class Planta {
2     protected:
3         int  aguaAcumulada;
4         int  nutrientesAcumulados;
5         Beleza beleza;  // FEIA, NEUTRA, BONITA
6
7     public:
8         virtual void avancaInstante(...) = 0;
9         virtual bool deveMorrer() const = 0;
10        virtual void aoMorrer(Posicao&) = 0;
11        virtual char getSimbolo() const = 0;
12 };
    
```

Listing 2.1: Hierarquia de Planta

Classes derivadas:

- Cacto — planta neutra, absorve 25% água do solo;
- Roseira — planta bonita, requer cuidados constantes;
- ErvaDaninha — planta feia, multiplica-se agressivamente;
- PlantaExotica — comportamento aleatório/híbrido.

Ferramentas

```

1 class Ferramenta {
2 protected:
3     int numeroSerie;      // ID unico
4     int capacidadeAtual; // Quantidade restante
5
6 public:
7     virtual char getSimbolo() const = 0;
8     virtual void aplicar(Posicao* pos) = 0;
9 };

```

Listing 2.2: Hierarquia de Ferramenta

Classes derivadas:

- Regador (g) — adiciona água ao solo;
- PacoteAdubo (a) — adiciona nutrientes ao solo;
- TesouraPoda (t) — remove plantas feias;
- FerramentaZ (z) — super fertilizador.

2.2 Estruturas de Dados Principais

2.2.1 Jardim: grelha 2D sem `std::vector`

Restrição do enunciado: não é permitido usar `std::vector` para armazenar as posições do solo.

Implementação

```

1 class Jardim {
2     Posicao** grelha; // Array 2D dinamico
3     int linhas;
4     int colunas;
5
6 public:
7     Jardim(int l, int c) {
8         grelha = new Posicao*[linhas];
9         for (int i = 0; i < linhas; i++) {
10             grelha[i] = new Posicao[colunas];
11         }
12     }
13 };

```

```

14 ~Jardim() {
15     for (int i = 0; i < linhas; i++) {
16         delete[] grelha[i];
17     }
18     delete[] grelha;
19 }
20 };

```

Listing 2.3: Estrutura do Jardim

Justificação:

- Respeita a restrição do enunciado;
- Acesso $O(1)$: `grelha[linha][coluna]`;
- Memória contígua por linha (boa localidade de cache);
- Usa apenas a memória necessária (sem overhead de vector).

2.2.2 Posição: célula individual

```

1 class Posicao {
2     int agua;
3     int nutrientes;
4     Planta* planta;           // Max 1 planta
5     Ferramenta* ferramenta;   // Max 1 ferramenta
6
7 public:
8     ~Posicao() {
9         delete planta;
10        delete ferramenta;
11    }
12
13    // Prevenir copia (evita double-delete)
14    Posicao(const Posicao&) = delete;
15    Posicao& operator=(const Posicao&) = delete;
16 };

```

Listing 2.4: Classe Posicao

Ownership: a `Posicao` é responsável por destruir as plantas e ferramentas que contém.

2.2.3 Gestão de memória com smart pointers

```

1 class Simulator {
2     std::unique_ptr<Jardim> jardim;

```

```
3     std::unique_ptr<Jardineiro> jardineiro;  
4  
5     ~Simulator() = default;  
6 };
```

Listing 2.5: Simulator - Gestão Segura

Vantagens:

- Sem *memory leaks*;
- *Exception-safe*;
- Ownership claro;
- Código mais limpo.

3 IMPLEMENTAÇÃO, DESIGN E VALIDAÇÃO

3.1 Implementação das Funcionalidades

3.1.1 Loop principal: avanca()

O método `avanca()` é o coração do simulador:

```

1 void Simulator::avanca(int numInstantes) {
2     for (int i = 0; i < numInstantes; i++) {
3         resetaContadoresTurno();
4         jardim->avancaInstante();
5         aplicaFerramentaAtiva();
6
7         for (int l = 0; l < jardim->getLinhas(); l++) {
8             for (int c = 0; c < jardim->getColunas(); c++) {
9                 Posicao* pos = jardim->getPosicao(l, c);
10                if (pos && pos->temPlanta()) {
11                    Planta* planta = pos->getPlanta();
12                    planta->avancaInstante(*pos, *jardim, l, c);
13                }
14            }
15        }
16
17        verificaMortes();
18        processaMultiplicacoes();
19        renderizaJardim();
20    }
21 }

```

Listing 3.1: Loop Principal do Simulador

3.1.2 Comportamento das plantas

Cacto

- **Absorção:** 25% da água do solo, até 5 nutrientes;
- **Morte:** água do solo > 100 por 3 turnos **ou** nutrientes = 0 por 3 turnos;
- **Multiplicação:** nutrientes > 100 e água > 50.

Roseira

- **Consumo:** perde 4 água e 4 nutrientes por turno;
- **Absorção:** 5 água e 8 nutrientes do solo;
- **Morte:** água < 1, nutrientes < 1, nutrientes > 199, ou cercada;
- **Multiplicação:** nutrientes > 100.

Erva Daninha

- **Absorção:** 1 água e 1 nutriente por turno;
- **Morte:** após 60 instantes;
- **Multiplicação:** a cada 5 turnos se nutrientes > 30, **mata** planta vizinha.

3.2 Ferramentas

3.2.1 Factory Pattern

```

1 Ferramenta* Simulator::criaFerramenta(char tipo) const {
2     static int proximoNumeroSerie = 1;
3
4     switch (tipo) {
5         case 'g': return new Regador(proximoNumeroSerie++);
6         case 'a': return new PacoteAdubo(proximoNumeroSerie++);
7         case 't': return new TesouraPoda(proximoNumeroSerie++);
8         case 'z': return new FerramentaZ(proximoNumeroSerie++);
9         default: return nullptr;
10    }
11 }

```

Listing 3.2: Criação de Ferramentas

Número de série: a variável static garante IDs únicos e crescentes.

3.2.2 Aplicação de ferramentas (polimorfismo)

```

1 void Regador::aplicar(Posicao* pos) {
2     if (capacidadeAtual >= 10) {
3         pos->adicionaAgua(10);
4         capacidadeAtual -= 10;
5     }
6 }
7

```

```

8 void TesouraPoda::aplicar(Posicao* pos) {
9     if (pos->temPlanta() &&
10         pos->getPlanta()->getBeleza() == Beleza::FEIA) {
11         Planta* p = pos->removePlanta();
12         p->aoMorrer(*pos);
13         delete p;
14     }
15 }

```

Listing 3.3: Polimorfismo em Ação

3.3 Limites por turno

Ação	Limite	Implementação
Colher plantas	5	plantasColhidasTurno
Plantar plantas	2	plantasPlantadasTurno
Movimentos	10	jardineiro->movimentosTurno
Entrar/Sair	1	entrouEsteTurno / saiuEsteTurno

Tabela 3.1: Limites por Turno

3.4 Decisões de design

3.4.1 Padrões de design

Factory Method

- **Onde:** `criaPlanta()`, `criaFerramenta()`;
- **Porquê:** centraliza criação de objetos e facilita manutenção;
- **Benefício:** adicionar nova planta/ferramenta implica alteração localizada.

Template Method

- **Onde:** classe abstrata `Planta`;
- **Porquê:** define o esqueleto do comportamento, com detalhes nas subclasses.

Strategy

- **Onde:** `Ferramenta::aplicar()`;
- **Porquê:** cada ferramenta encapsula uma estratégia distinta.

3.4.2 Princípios SOLID (síntese)

- **SRP:** Posicao, Jardim, Simulator e Interface têm responsabilidades bem delimitadas;
- **OCP/LSP:** novas plantas/ferramentas podem ser adicionadas respeitando as abstrações;
- **DIP:** dependência sobre interfaces abstratas (Planta, Ferramenta).

3.5 Renderização do jardim

Quando múltiplos objetos ocupam a mesma posição, a prioridade é:

1. Jardineiro (*);
2. Planta (c, r, e, x);
3. Ferramenta (g, a, t, z);
4. Espaço vazio.

```

1 char Jardim::getCaracter(int linha, int col,
2                           bool temJardineiro,
3                           int linhaJard, int colJard) const {
4     if (temJardineiro && linha == linhaJard && col == colJard) {
5         return '*';
6     }
7
8     const Posicao* pos = getPosicao(linha, col);
9
10    if (pos->temPlanta()) {
11        return pos->getPlanta()->getSimbolo();
12    }
13    if (pos->temFerramenta()) {
14        return pos->getFerramenta()->getSimbolo();
15    }
16    return ' ';
17 }

```

Listing 3.4: Algoritmo de Renderização

3.6 Testes e validação

3.6.1 Compilação

- **Compilador:** Apple Clang 17.0 (macOS);

- **Standard:** C++17;
- **Build system:** CMake 3.20+.

3.6.2 Casos de teste (exemplos)

Teste 1: criação e renderização

```
1 jardim 5 5
```

Teste 2: plantar e crescer

```
1 planta aa c
2 planta bb r
3 planta cc e
4 avanca 10
5 lplantas
```

3.6.3 Validações implementadas

Validação	Mensagem de Erro
Jardim não existe	"Erro: Jardim nao existe!"
Posição inválida	"Erro: Posicao invalida!"
Limite de colheitas	"Ja colheu 5 plantas neste turno!"
Limite de plantações	"Ja plantou 2 plantas neste turno!"
Limite de movimentos	"Limite de movimentos atingido!"
Posição ocupada	"Ja existe uma planta nessa posicao!"
Ferramenta não encontrada	"Ferramenta #X nao encontrada"

Tabela 3.2: Validações e Mensagens de Erro

3.7 Estatísticas e estado da Meta 2

3.7.1 Métricas de código (aprox.)

Métrica	Valor
Classes	18
Ficheiros .h	18
Ficheiros .cpp	18
Linhas de código	2800
Métodos implementados	120+
Comandos disponíveis	20

Tabela 3.3: Estatísticas do Código

3.7.2 Funcionalidades implementadas

- ✓ Loop principal (`avanca()`);
- ✓ Comportamento das 4 plantas;
- ✓ Sistema de mortes;
- ✓ Aplicação de ferramentas;
- ✓ Movimento do jardineiro;
- ✓ Limites de ações por turno e renderização completa.

3.8 Dificuldades e melhorias

3.8.1 Dificuldades encontradas (síntese)

- Implementação da grelha dinâmica 2D sem `std::vector`;
- Definição consistente de ownership para evitar *double delete*;
- Integração de múltiplos limites por turno com mensagens de erro claras.

3.8.2 Melhorias futuras

- Concluir e testar a multiplicação de plantas;
- Implementar *save/load* com cópia profunda;
- Adicionar testes unitários e documentação Doxygen;
- Explorar uma interface gráfica.

4 CONCLUSÃO

O projeto **Simulador de Jardim** foi implementado com sucesso, cumprindo os requisitos da Meta 2.

4.1 Objetivos alcançados

- ✓ Aplicação correta de POO (herança, polimorfismo, encapsulamento);
- ✓ Gestão segura de memória dinâmica;
- ✓ Respeito às restrições do enunciado;
- ✓ Funcionalidades principais implementadas e validadas.

4.2 Aprendizagens

Este trabalho permitiu consolidar conhecimentos em:

- Design de hierarquias de classes;
- Padrões de design (Factory, Template Method, Strategy);
- Gestão manual de memória vs. smart pointers;
- Importância de ownership claro e validação de input.

4.3 Palavras finais

O simulador está funcional e pronto para demonstração. A arquitetura escolhida facilita manutenção e extensões futuras, permitindo adicionar novos tipos de plantas e ferramentas com impacto mínimo no código existente.

REFERÊNCIAS BIBLIOGRÁFICAS

- [1] Enunciado do Trabalho Prático – Programação Orientada a Objetos, ISEC, ano letivo 2025/2026.

ANEXOS

Comando	Descrição
jardim <l> <c>	Cria jardim com l linhas e c colunas
avanca [n]	Avança n instantes (padrão: 1)
lplantas	Lista todas as plantas
lplanta <pos>	Info de planta específica
larea	Lista posições não vazias
lsolo <pos> [raio]	Info do solo com raio opcional
lferr	Lista ferramentas do jardineiro
colhe <pos>	Colhe planta (max 5/turno)
planta <pos> <tipo>	Planta (c/r/e/x, max 2/turno)
compra <tipo>	Compra ferramenta (g/a/t/z)
pega <num>	Pega ferramenta pelo número
larga	Larga ferramenta da mão
e/d/c/b	Move jardineiro
entra <pos>	Entra no jardim
sai	Sai do jardim
grava <nome>	Grava estado (TODO)
recupera <nome>	Recupera estado (TODO)
apaga <nome>	Apaga estado (TODO)
executa <ficheiro>	Executa comandos de ficheiro
fim	Termina simulador

Tabela 4.1: Lista completa de comandos

```
SimGarden/  
|-- CMakeLists.txt  
|-- main.cpp  
|-- config/  
|   |-- Settings.h  
|   '-- RandomGenerator.h  
|-- core/  
|   |-- Simulator.h/cpp  
|   |-- Interface.h/cpp  
|   '-- Validator.h/cpp  
|-- jardim/  
|   |-- Jardim.h/cpp  
|   '-- Posicao.h/cpp  
|-- jardineiro/  
|   '-- Jardineiro.h/cpp  
|-- plantas/  
|   |-- Planta.h/cpp  
|   |-- Cacto.h/cpp  
|   |-- Roseira.h/cpp  
|   |-- ErvaDaninha.h/cpp  
|   '-- PlantaExotica.h/cpp  
'-- ferramentas/  
    |-- Ferramenta.h/cpp  
    |-- Regador.h/cpp  
    |-- PacoteAdubo.h/cpp  
    |-- TesouraPoda.h/cpp  
    '-- FerramentaZ.h/cpp
```



**Instituto Superior
de Engenharia**

Politécnico de Coimbra